# NLU Course Project

*Martina Panini (257851)*

University of Trento

martina.panini@studenti.unitn.it

## 1. Introduction

In this project, I address the joint tasks of improve model performance on intent classification and slot filling on the ATIS dataset. In First Part, I incrementally improve a baseline IAS model by adding bidirectionality to its recurrent encoder and introducing dropout to its hidden representations. Then, for the Second Part, I replace the custom encoder with a pre-trained BERT backbone, fine-tuning it in a multi-task setup [1]: a token-level head for slot tagging and a sentence-level head for intent classification. Throughout, I tune learning rates to maximize F1 and accuracy.

## 2. Implementation details

**First Part.** I started from a baseline architecture `ModelIAS`, which uses a unidirectional LSTM.

To improve performance, I incrementally introduced two key enhancements: *bidirectionality* and *dropout*.
A bidirectional LSTM processes the input sequence in two directions: one from left-to-right and another from right-to-left. The hidden states from both directions are then concatenated. This can improve performance both for slot filling and intent classification because bidirectionality allows the model to capture context from both past and future words in an utterance. For slot filling, knowing what comes after a word can be as important as knowing what came before it to correctly classify a slot. For intent classification, the overall meaning often relies on information spread across the entire utterance.

Next, I added a dropout layer with a dropout probability of $p = 0.5$ after the LSTM output. Dropout is a regularization technique where, during training, a certain percentage (dropout probability) of neurons are randomly "dropped out" (their outputs are set to zero) at each update. This is done to avoid overfitting.

Both enhancements were implemented via optional flags (`isBidirectional` and `isDropout`) in the model's constructor, enabling controlled ablation studies to isolate the impact of each modification.

**Second Part.** In this part, I adapted the existing codebase to fine-tune a pre-trained BERT model (BERT-base) jointly on intent classification and slot filling, following the multi-task approach proposed by Chen et al. [1].
A key challenge addressed was the handling of BERT's sub-tokenization. Since BERT may split a single word into multiple sub-tokens, this will cause a desalination on the length of the utterances and the one of the slots.

First of all, the `Lang` class no longer need to map word to IDs because `BertTokenizer` is passed directly to the dataset. Now this class is used only to map labels.
Then, dataset class had to handle `[CLS]` and `[SEP]` tokens, that are explicitly added to the beginning and end of each utterance. Padding tokens are added to the beginning and end of the slot sequence to match the `[CLS]` and `[SEP]` tokens added to the utterance.
Key changes are made in the function that map sequences into IDS to handle BERT's subword tokenization. BERT's WordPiece tokenizer can split single words into multiple sub-tokens. The approach iterates through each original word of the utterance. For each word, BERT tokenizer is used to get its sub-tokens. The original slot label is assigned only to the first sub-token of a word; subsequent sub-tokens from that same word receive a `'pad'` label.

This ensures slot labels align with BERT's sub-tokens for training. During evaluation, these extra 'pad' labels are ignored when calculating the F1 score, maintaining accurate performance measurement despite subword tokenization.

On the modeling side, I introduced a new ModelBert class which uses the pre-trained BERT model. This replaces the `embedding` and `LSTM` layers of the previous model.Then are added two task-specific classification layers: one for slot filling (token-level, BERT's `last_hidden_state` provides a contextualized representation for each input token, which is then fed into this linear layer to classify the slot for that token.) and one for intent classification (sentence-level). The `[CLS]` token's final hidden state is conventionally used as the aggregate representation of the input sequence for classification tasks in BERT. The forward pass was adapted to support BERT inputs such as attention masks and token type IDs. Losses from both tasks were combined and backpropagated jointly.

## 3. Results

This section presents the evaluation results for both parts of the project on the ATIS test set. The performance of the models is assessed using two standard metrics: Intent Classification Accuracy and Slot Filling F1 score.

For the first part of the project, was incrementally improved the baseline IAS model, the experiments were conducted over 5 independent runs. The final reported metrics for each configuration represent the average performance across these runs, providing a more robust estimate of the model's capabilities and the impact of each modification. . Results are summarized in Table 1.
Adding a Bidirectional LSTM increased both Intent Accuracy (from $93.2\%$ to $94.2\%$) and Slot Filling F1 (from $90.3\%$ to $93.8\%$) by capturing richer context. Further adding a Dropout layer improved performance even more, reaching $95.0\%$ Intent Accuracy and $94.1\%$ Slot F1, demonstrating its effectiveness in preventing overfitting and improving generalization.

For the second part of the project, I fine-tuned a pre-trained BERT-base model in a multi-task setting for joint intent classification and slot filling. This experiment was conducted as a single run. Table 2 presents the results obtained with this configuration.

The results clearly demonstrate that fine-tuning a pre-trained transformer model is a superior approach for this NLU task compared to training a custom recurrent network architecture from scratch.

| Configuration | Intent Acc. (%) | Slot F1 |
|---|---|---|
| Baseline IAS model | 0.932 | 0.903 |
| + Bidirectional LSTM | 0.942 | 0.938 |
| + Dropout | **0.95** | **0.941** |

Table 1: *First Part: Impact of bidirectionality and dropout.*

| Configuration | Intent Acc. (%) | Slot F1 |
|---|---|---|
| BERT joint fine-tuning | **0.978** | **0.955** |

Table 2: *Second Part: Performance of the fine-tuned BERT model.*

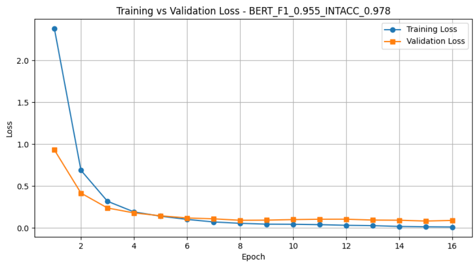


Figure 1: *Loss plot of the best performance of Second Part*

# 4. References

[1] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.